

Tích hợp Chế độ Auto (SCL) vào OB1 (Ladder)

Hướng dẫn chi tiết — Dự án KLTN PLC_1

1. Tổng quan kiến trúc

OB1 (Ladder) đóng vai trò khung chính (chiếc xe). FB_Auto_Main (SCL) là một khối chức năng con (hệ thống phanh) — được OB1 gọi như một thành phần, không can thiệp vào logic điều khiển khác.

Nguyên tắc cốt lõi:

- Manual FB2 và Auto FB_Auto_Main đều ghi ra biến trung gian trong Flags [DB4], KHÔNG ghi thẳng ra Q.
- Một Network riêng cuối OB1 mới ghi từ Flags ra Q0.0/Q0.1/Q0.2 — đảm bảo chỉ 1 nguồn ghi mỗi output.
- SCL tự quản lý TriggerReq, FIFO, state machine — OB1 chỉ truyền input/output.

2. Thay đổi trong Flags [DB4]

Thêm 6 biến mới vào cuối DB4 (sau DebugState):

Tên biến	Kiểu	Mục đích
Auto_Conveyor	Bool	Output trung gian Conveyor từ chế độ Auto
Auto_Cylinder1	Bool	Output trung gian Cylinder1 từ chế độ Auto
Auto_Cylinder2	Bool	Output trung gian Cylinder2 từ chế độ Auto
Manual_Conveyor	Bool	Output trung gian Conveyor từ chế độ Manual
Manual_Cylinder1	Bool	Output trung gian Cylinder1 từ chế độ Manual
Manual_Cylinder2	Bool	Output trung gian Cylinder2 từ chế độ Manual

3. Tạo mới FB_Auto_Main (SCL) — v0.3

Tạo Function Block mới trong TIA Portal, đặt tên FB_Auto_Main, ngôn ngữ SCL. Tạo instance DB mới: DB_AutoMain [DB6]. Dán toàn bộ code sau:

```
FUNCTION_BLOCK FB_Auto_Main
{ S7_Optimized_Access := 'TRUE' }
VERSION : 0.3

VAR_INPUT
    SS0          : BOOL;
    SS1          : BOOL;
    SS2          : BOOL;
    PC_DataReady : BOOL;
```

```

        PC_KetQua      : INT;
        ResetFault     : BOOL;
END_VAR

VAR_OUTPUT
    Conveyor          : BOOL;
    Cylinder1          : BOOL;
    Cylinder2          : BOOL;
    PLC_TriggerReq     : BOOL;
    ErrorActive        : BOOL;
    ErrorCode          : INT;
    QueueCount         : INT;
    DebugState         : INT;
END_VAR

VAR
    prevSS0 : BOOL; prevSS1 : BOOL; prevSS2 : BOOL;
    ss0Rise : BOOL;  ss0Fall : BOOL;
    ss1Rise : BOOL;  ss2Rise : BOOL;
    triggerCycleActive : BOOL;
    aiConsumedThisCycle : BOOL;
    fifo      : FB_FIFO10;
    headValue : INT;
    fifoOverflow : BOOL; fifoUnderflow : BOOL;
    pushReq    : BOOL; pushResult : INT; popReq : BOOL;
    cyl1SeqState : INT; cyl2SeqState : INT;
    cyl1Done    : BOOL; cyl2Done   : BOOL;
    cyl1Busy    : BOOL; cyl2Busy   : BOOL;
    actCyl1Req  : BOOL; actCyl2Req : BOOL;
    CylTimer1   : TON; CylTimer2   : TON;
    errOverflow : BOOL;
    errInvalidResult : BOOL;
    state : INT;
END_VAR

VAR CONSTANT
    ST_IDLE      : INT := 0;
    ST_RUN       : INT := 10;
    ST_WAIT_SS2  : INT := 20;
    ST_ACT_CYL1  : INT := 30;
    ST_ACT_CYL2  : INT := 40;
    ST_FAULT     : INT := 90;
END_VAR

BEGIN
    // ---- 1) Bắt cạnh ----
    ss0Rise := SS0 AND NOT prevSS0;
    ss0Fall := NOT SS0 AND prevSS0;
    ss1Rise := SS1 AND NOT prevSS1;
    ss2Rise := SS2 AND NOT prevSS2;
    prevSS0 := SS0; prevSS1 := SS1; prevSS2 := SS2;

    // ---- 2) AI Handshake ----
    pushReq := FALSE; errOverflow := FALSE; errInvalidResult := FALSE;

    IF ss0Rise THEN
        IF QueueCount >= 10 THEN
            errOverflow := TRUE;
        ELSE
            PLC_TriggerReq      := TRUE;
            triggerCycleActive  := TRUE;
            aiConsumedThisCycle := FALSE;
        END_IF;
    END_IF;
END_VAR

```

```

    IF PC_DataReady AND (NOT aiConsumedThisCycle) AND triggerCycleActive
THEN
    IF (PC_KetQua < 0) OR (PC_KetQua > 4) THEN
        errInvalidResult := TRUE;
    ELSE
        pushReq           := TRUE;
        pushResult        := PC_KetQua;
        aiConsumedThisCycle := TRUE;
    END_IF;
END_IF;

IF triggerCycleActive AND ss0Fall THEN
    PLC_TriggerReq      := FALSE;
    triggerCycleActive := FALSE;
    aiConsumedThisCycle := FALSE;
END_IF;

// ---- 3) State machine ----
popReq := FALSE;  actCyl1Req := FALSE;  actCyl2Req := FALSE;

CASE state OF
    ST_IDLE:
        Conveyor := FALSE;

    ST_RUN:
        Conveyor := TRUE;
        IF ss1Rise THEN
            IF headValue = 2 THEN
                actCyl1Req := TRUE;  state := ST_ACT_CYL1;
            ELSE
                state := ST_WAIT_SS2;
            END_IF;
        END_IF;

    ST_WAIT_SS2:
        Conveyor := TRUE;
        IF ss2Rise THEN
            IF (headValue = 3) OR (headValue = 4) THEN
                actCyl2Req := TRUE;  state := ST_ACT_CYL2;
            ELSE
                popReq := TRUE;  state := ST_RUN;
            END_IF;
        END_IF;

    ST_ACT_CYL1:
        Conveyor := FALSE;
        IF cyl1Done THEN popReq := TRUE;  state := ST_RUN;  END_IF;

    ST_ACT_CYL2:
        Conveyor := FALSE;
        IF cyl2Done THEN popReq := TRUE;  state := ST_RUN;  END_IF;

    ST_FAULT:
        Conveyor := FALSE;  PLC_TriggerReq := FALSE;
        IF ResetFault THEN state := ST_IDLE;  END_IF;

ELSE state := ST_IDLE;
END_CASE;

// ---- 4) Cylinder sequences ----
FC_Cylinder1_Seq(
    StartAction := actCyl1Req,
    SeqState    := cyl1SeqState,
    CylTimer    := CylTimer1,
    Cylinder1Cmd => Cylinder1,
    Busy        => cyl1Busy,

```

```

        DonePulse    => cyl1Done
    );
    FC_Cylinder2_Seq(
        StartAction  := actCyl2Req,
        SeqState     := cyl2SeqState,
        CylTimer     := CylTimer2,
        Cylinder2Cmd => Cylinder2,
        Busy         => cyl2Busy,
        DonePulse    => cyl2Done
    );

    // ---- 5) FIFO ----
    fifo(
        Push        := pushReq,
        PushValue   := pushResult,
        Pop         := popReq,
        HeadValue   => headValue,
        Count       => QueueCount,
        Overflow    => fifoOverflow,
        Underflow   => fifoUnderflow
    );

    // ---- 6) Fault ----
    ErrorActive := FALSE;  ErrorCode := 0;
    IF errOverflow OR fifoOverflow THEN
        ErrorActive := TRUE;  ErrorCode := 301;
    ELSIF fifoUnderflow THEN
        ErrorActive := TRUE;  ErrorCode := 302;
    ELSIF errInvalidResult THEN
        ErrorActive := TRUE;  ErrorCode := 401;
    END_IF;

    IF ErrorActive THEN state := ST_FAULT; END_IF;
    DebugState := state;
END_FUNCTION_BLOCK

```

4. Sửa OB1 Ladder — từng Network

Network 1 — Giữ nguyên

Không thay đổi. Logic Start-Stop và Lamp vẫn hoạt động bình thường.

Network 2 — Giữ nguyên

Không thay đổi. Logic chọn chế độ Auto/Manual.

Network 3 — Sửa: xóa dòng TriggerReq

Xóa dòng: I0.3 (SS0) → DB2.DBX0.3 (PLC_TriggerReq)

Lý do: SCL FB_Auto_Main tự quản lý TriggerReq theo handshake. Nếu giữ dòng này, TriggerReq sẽ luôn = SS0 và xung đột với logic handshake của SCL.

Giữ lại 2 dòng còn lại:

- I0.4 (SS1) → DB2.DBX0.4 (PLC_Sensor1)
- I0.5 (SS2) → DB2.DBX0.5 (PLC_Sensor2)

Network 4 — Sửa: Manual output ra Flags thay vì Q trực tiếp

Điều kiện EN giữ nguyên: DB2.DBX0.1 (PLC_Manual) AND I0.2 (btManual)

Sửa mapping output của FB2 Mode_Manual:

Output FB2	Ghi vào
conveyor	"Flags".Manual_Conveyor (thay vì Q0.0)
cylinder_1	"Flags".Manual_Cylinder1 (thay vì Q0.1)
cylinder_2	"Flags".Manual_Cylinder2 (thay vì Q0.2)

Network 5 — MỚI: Gọi FB_Auto_Main

Điều kiện EN: DB2.DBX0.2 (PLC_Running) AND I0.1 (btAuto)

Tạo instance DB_AutoMain [DB6] cho FB_Auto_Main. Mapping input/output:

Pin FB_Auto_Main	Hướng	Kết nối
SS0	← Input	I0.3
SS1	← Input	I0.4
SS2	← Input	I0.5
PC_DataReady	← Input	"DB_GET".PC_DataReady
PC_KetQua	← Input	"DB_GET".PC_KetQua
ResetFault	← Input	"Flags".ResetFault
Conveyor	→ Output	"Flags".Auto_Conveyor
Cylinder1	→ Output	"Flags".Auto_Cylinder1
Cylinder2	→ Output	"Flags".Auto_Cylinder2
PLC_TriggerReq	→ Output	"DB_PUT".PLC_TriggerReq
ErrorActive	→ Output	"Flags".ErrorActive
ErrorCode	→ Output	"Flags".ErrorCode
QueueCount	→ Output	"Flags".QueueCount
DebugState	→ Output	"Flags".DebugState

Network 6 — MỚI: Output thật ra Q

Network này là nguồn DUY NHẤT ghi ra Q. Dùng tiếp điểm OR:

Nguồn Auto	Nguồn Manual	Output Q
"Flags".Auto_Conveyor	"Flags".Manual_Conveyor	Q0.0 (Conveyor)
"Flags".Auto_Cylinder1	"Flags".Manual_Cylinder1	Q0.1 (Cylinder1)
"Flags".Auto_Cylinder2	"Flags".Manual_Cylinder2	Q0.2 (Cylinder2)

Cách viết trong Ladder (ví dụ Conveyor):

```
"Flags".Auto_Conveyor  --|  |--+
                        +----- Q0.0 (Conveyor)
"Flags".Manual_Conveyor--|  |--+
```

Làm tương tự cho Cylinder1 và Cylinder2.

5. Checklist thực hiện theo thứ tự

1. Mở Flags [DB4] → thêm 6 biến mới (Auto_Conveyor, Auto_Cylinder1, Auto_Cylinder2, Manual_Conveyor, Manual_Cylinder1, Manual_Cylinder2) kiểu Bool vào cuối.
2. Tạo FB_Auto_Main mới (SCL) → paste code Section 3 → compile.
3. Tạo instance DB_AutoMain [DB6] từ FB_Auto_Main.
4. OB1 Network 3: xóa dòng I0.3 → PLC_TriggerReq.
5. OB1 Network 4: sửa output Manual FB2 ghi vào Flags thay vì Q.
6. OB1 thêm Network 5: gọi FB_Auto_Main với điều kiện và mapping như bảng.
7. OB1 thêm Network 6: 3 coil Q0.0/Q0.1/Q0.2 với 2 tiếp điểm OR (Auto + Manual).
8. Compile toàn bộ → Download → Test chế độ Manual trước, sau đó Auto.

6. Mã trạng thái Auto để quan sát (DebugState)

Giá trị	Tên state	Ý nghĩa
0	ST_IDLE	Chờ — chưa nhấn Start hoặc đã dừng
10	ST_RUN	Đang chạy — băng tải ON, chờ sản phẩm
20	ST_WAIT_SS2	Sản phẩm qua SS1 (không NG_L), chờ SS2
30	ST_ACT_CYL1	Đang kích Cylinder1 (loại NG_L tại SS1)
40	ST_ACT_CYL2	Đang kích Cylinder2 (loại NG_H/MISSING tại SS2)
90	ST_FAULT	Lỗi — băng tải dừng, chờ ResetFault

7. Mã lỗi (ErrorCode)

ErrorCode	Nguyên nhân
301	FIFO overflow — hơn 10 sản phẩm chưa xử lý trên băng tải
302	FIFO underflow — pop khi FIFO rỗng (logic sai)
401	Kết quả AI ngoài khoảng 0..4 (PC gửi sai)